

腾讯云智能渗透挑战赛 Agent 架构分析

原创 fearless123 SecureNexusLab 2026年5月26日 09:39 北京

前言



两个项目均来自第二届腾讯云黑客松智能渗透挑战赛。BreachWeave 获一等奖（1/613），Cairn 是唯一 AK 的队伍（54/54）。本文从架构设计视角梳理两者的 Agent 协作机制，并就分析输出一下个人的看法和思路。

一、BreachWeave: Manager / Solver / Observer 三角色协作

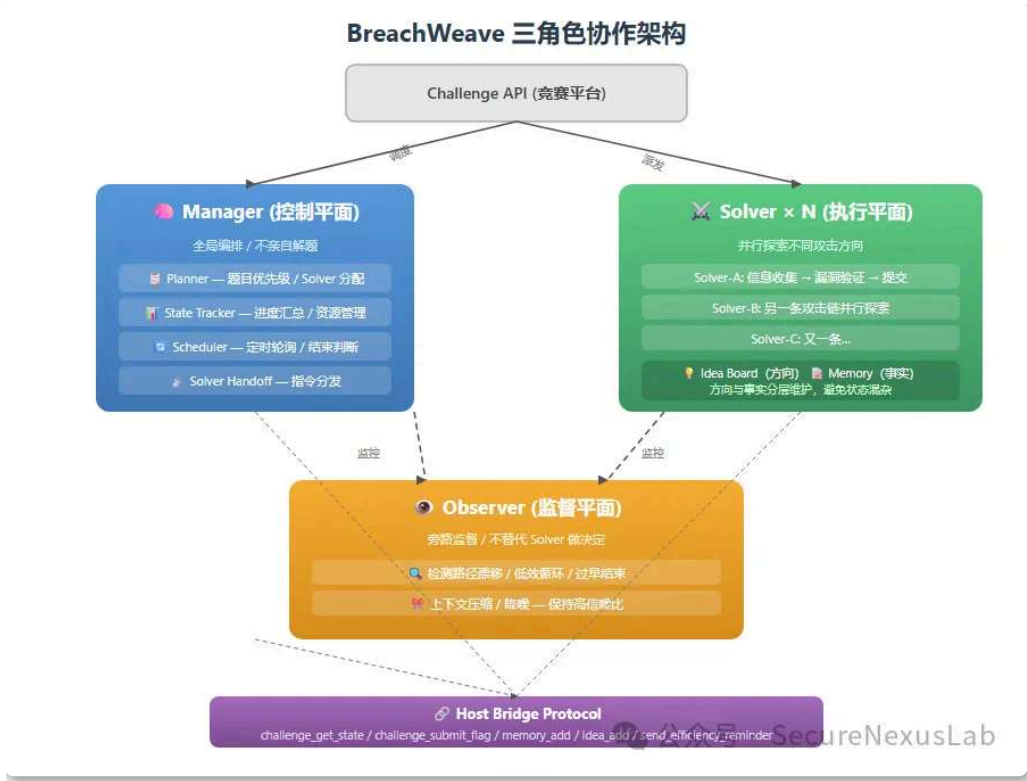
1.1 核心理念

BreachWeave 的设计哲学是「角色分工」。它把渗透过程拆成三种职责，交给三个不同角色的 Agent 去做：

- 「Manager」 — 控制平面，不亲自解题，只做全局调度
- 「Solver」 — 执行平面，真正跑 nmap、写 exploit、提交 flag
- 「Observer」 — 监督平面，旁路纠偏，防止路径漂移

三者各司其职，通过 Host Bridge 协议交互。

1.2 架构总览



1.3 关键设计细节

「**状态分层**」：系统把 "我接下来该往哪走" (Idea) 和 "我已经确认了什么" (Memory) 严格分开。Idea 有状态机 (pending → testing → verified/failed/skipped)，Memory 按类型分类 (fact / evidence / failure / note / hint)。这让决策始终保持在高信噪比上。

「**Planner 调度策略**」：Manager 里有一个 Planner prompt，它每隔 30 秒轮询一次状态快照，决定是否启动新 Solver、停止旧 Solver、或保持不动。核心原则是 "稳定优先于频繁变更"——它不会为了看起来更均衡而去调换已有阵型。

「**Observer 纠偏**」：Observer 通过 Host Bridge 的 `send_efficiency_reminder` 向 Solver 发送轻量提醒，例如 "这个方向已经试过了，建议转向"。它不做决策，只做纠偏。

「**上下文压缩**」：Solver 每轮拿到的不是原始数据，而是压缩后的摘要——Idea 取 top 6、Memory 取最近 6 条、handoff 限 900 字符。这解决了长任务中上下文不断膨胀的经典问题。

二、Cairn：黑板架构与状态空间搜索

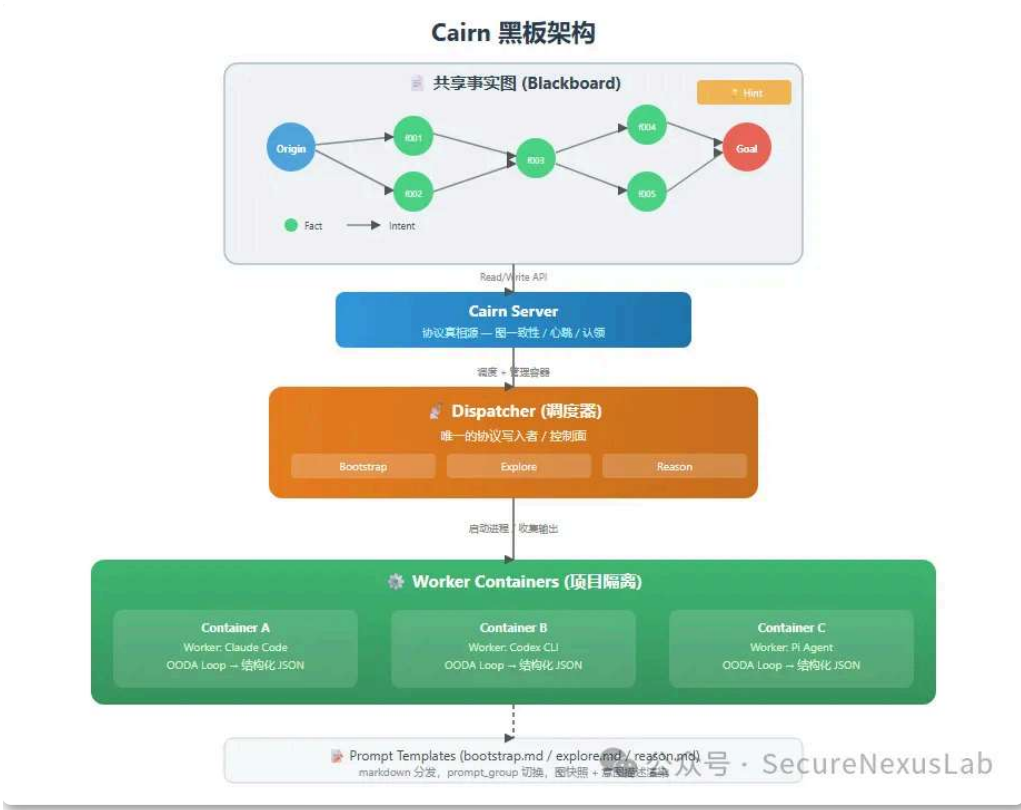
2.1 核心理念

Cairn 不定义角色、不预设工作流。它把渗透测试抽象为一个「**状态空间搜索问题**」：

- 「**起点**」 (Origin)：已知目标
- 「**终点**」 (Goal)：拿到 shell / flag
- 「**路径**」 (Path)：未知，需要搜索

整个系统只维护一张有向无环图 (DAG)，由 Fact (事实) 和 Intent (意图) 组成。Agent 通过读图、写图来间接协调——这是蚁群信息素 (Stigmergy) 的数字化版本。

2.2 架构总览



2.3 关键设计细节

「三个原语就够了」:

「概念」	「含义」
「Fact」	已确认的客观事实
「Intent」	已声明的探索方向（有状态：open → claimed → concluded）
「Hint」	外部注入的人类判断

「三种任务，同一个 Worker」:

「任务」	「触发时机」	「输入」	「输出」
「Bootstrap」	项目刚创建	Origin + Goal + Hint	Fact + 可能的 Complete
「Explore」	有未认领的 Intent	图快照 + 当前 Intent	一个 Fact
「Reason」	需要全局判断	完整图 + 所有 Intent	Complete / 新 Intent / noop

「Dispatcher 是唯一的控制面」: Agent 不直接认领 Intent、不直接心跳、不直接调用 API。一切都由 Dispatcher 代劳。Agent 只接收渲染好的 prompt，返回结构化 JSON。

「超时与收尾」: 每种任务都支持"第一阶段执行 + timeout/parse-fail 后用同一 session 进入 conclude 收尾"的双阶段模式。这保证即使超时，也能产出至少一个 Fact。

三、两者对比

设计理念对比



容易看出二者差别较明显，并且二者都取得了较好的效果

「维度」	「BreachWeave」	「Cairn」
设计哲学	角色分工、各司其职	无角色、图驱动
Agent 身份	Manager / Solver / Observer，有固定身份	无固定身份，同一个 Worker 执行三种任务
通信方式	Host Bridge 指令注入（Follow-up / Steer）	共享图读写（Stigmergy），零直接通信
状态结构	Idea（方向）+ Memory（事实），两套系统	Fact DAG + Intent 状态机，一张图
人类介入	User Strategy 字段注入策略	Hint 节点随时可插入
Worker 隔离	进程级，共享宿主机	Docker 容器级，项目隔离
可扩展性	SDK 层面，新增 prompt + skill	Worker adapter 层面，新增 driver

四、心得体会

4.1 "搜索"才是渗透测试的本质

两个项目最大的共识是：「**渗透测试不是执行脚本，而是搜索路径**」。BreachWeave 用 Idea 状态机表达"搜索方向的生命周期"，Cairn 用 Fact DAG 表达"搜索空间的生长过程"。两者殊途同归——都在解决同一个问题：如何让 AI Agent 在一个近无穷的状态空间里，高效地找到从起点到终点的路径，其重点在于「**如何高效的让ai在一定时间内不断的尝试不同的路径，不断试错最终得出正确的思路拿到flag**」，其中如何去尝试，并行还是串行；如何去反馈，此路不通还是此路通畅，是非常重要的设计要点。

4.2 "不让模型做它不擅长的事"

两者的共同点是「**把不该交给模型的决策收归系统**」：

- 结束条件不靠模型判断，BreachWeave 外置，Cairn 用 Reason 任务 + 图状态双重判断
- 调度逻辑不靠模型自由发挥，BreachWeave 的 Planner 有硬约束，Cairn 的 Dispatcher 是规则驱动
- 上下文大小靠系统控制，BreachWeave 有字符限制，Cairn 用图快照替代原始历史

模型只管做它最擅长的事：理解当前状态，决定下一步动作。

4.3 工程上的启发

对于想构建类似系统的开发者，有几个可借鉴的点：

1. 「**上下文管理比 prompt engineering 更重要**」。两个项目都不约而同地在上下文压缩上花了大量心思。
2. 「**结束条件必须外置**」。LLM 天然倾向于过早结束复杂任务，这是被反复验证的。
3. 「**并发不是越多越好**」。BreachWeave 限制最多 3 个 challenge 实例、7 个 solver；Cairn 按 Worker 的 `max_running` 控制。过度并发会导致资源浪费和上下文污染。

五、总结

两个项目代表了 AI 自动化渗透的两种主流范式：

- 「**BreachWeave**」 = 角色分工 + 指令驱动 + 状态分层 → 适合需要精确控制的竞赛场景
- 「**Cairn**」 = 黑板架构 + 图驱动 + 信息素协调 → 适合需要自由探索的开放场景

它们共同揭示了一个趋势：「**AI 自动化渗透的核心竞争力不在"用什么模型"，而在"怎么编排 Agent"**」。也就是说AI自动化渗透的要点在于如何去给不同的Agent分配任务以及不同的Agent之间应该如何去交互，解决了这两个问题就能设计出非常高效的AI自动化渗透项目。